

Desítková soustava (decimální)

- užívaná většinou lidí na naší planetě
- číslo je složeno z jednoho či více číslic
- číslice mohou nabývat hodnot:
 - 0, 1, 2, 3, 4, 5, 6, 7, 8, 9
- pokud by mělo dojít k „přetečení“ čísla přes hodnotu 9, přičteme +1 k číslici o jedna vyšší (carry)
 - $9 + 1 = 10$
 - $9 + 2 = 11$
 - $999 + 1 = 1000$

Dvojková soustava (binární)

- použita v naprosté většině elektroniky
- číslo je složeno z jednoho či více číslic
- číslice mohou nabývat hodnot:
 - 0, 1
- pokud by mělo dojít k „přetečení“ čísla přes hodnotu 1, přičteme +1 k číslici o jedna vyšší (carry)
 - $1 + 1 = 10$
 - $10 + 11 = 101$
 - $1111 + 1 = 10000$

Hlavní rozdíl mezi desítkovou a dvojkovou soustavou je počet hodnot jednotlivých číslic. Pro počítače je výhodné využít pouze 2 hodnot 0 či 1 (nesvítí / svítí; $<1,5V$ / $\geq 1,5V$). Rozlišování mezi více hodnotami je pro elektroniku velmi složité (využívá se například u SSD disků, kdy jedna buňka může nabývat několika hodnot, např.: $<0,2V$; $0,2 - 0,4V$; $0,4 - 0,8V$; $>0,8V \Rightarrow 0, 1, 2, 3$, ale takovéto hodnoty jsou vždy rychle převedeny na binární).

U desítkových číslic máme dány řády:

1. → jednotky
2. → desítky
3. → stovky
4. → tisíce
5. → deseti tisíce

U dvojkových číslic je vyšší řád vždy dvojnásobkem řádu předchozího:

1. → $2^0 = 1$
2. → $2^1 = 2$
3. → $2^2 = 4$
4. → $2^3 = 8$
5. → $2^4 = 16$

číslo 237 se tedy dá rozložit na $200 + 30 + 7$.

číslo 11101101 se dá rozložit na $1 \cdot 128 + 1 \cdot 64 + 1 \cdot 32 + 0 \cdot 16 + 1 \cdot 8 + 1 \cdot 4 + 0 \cdot 2 + 1 \cdot 1 = 237$

Kromě dvojkové a desítkové soustavy se můžeme setkat ještě s šestnáctkovou či osmičkovou soustavou. Těmi si ale dnes nebudeme plést hlavu, jen zmíním, že šestnáctková soustava je přímo konvertovatelná do binární a proto je používána při programování (číslice nabývají hodnot 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, a, b, c, d, e, f)

Porovnání čísel

dec	bin	dec	bin	dec	bin	dec	bin	dec	bin
0	0	10	1010	20	1 0100	30	1 1110	40	10 1000
1	1	11	1011	21	1 0101	31	1 1111	41	10 1001
2	10	12	1100	22	1 0110	32	10 0000	42	10 1010
3	11	13	1101	23	1 0111	33	10 0001	43	10 1011
4	100	14	1110	24	1 1000	34	10 0010	44	10 1100
5	101	15	1111	25	1 1001	35	10 0011	45	10 1101
6	110	16	1 0000	26	1 1010	36	10 0100	46	10 1110
7	111	17	1 0001	27	1 1011	37	10 0101	47	10 1111
8	1000	18	1 0010	28	1 1100	38	10 0110	48	11 0000
9	1001	19	1 0011	29	1 1101	39	10 0111	49	11 0001

Registry počítače

Registry počítače slouží k uložení čísel a hlavním parametrem je jejich bitová šířka, tj. kolik číslic hodnot 0 či 1 mohou uložit. Typické šířky jsou:

- 4bit – používalo se u prvních čipů pro kalkulačky, umí pracovat s čísly 0 – 15
- 8bit – první herní konzole a první PCčka (XT); pracují s čísly 0 - 255
- 16bit – PC/AT, zpravidla DOS či win 3.11; pracují s čísly 0 – 65535
- 32bit – éra windows 95 – win XP; pracují s čísly 0 – 4294967295
- 64bit – moderní PC; pracují s čísly 0 – 18446744073709551615
- 128/256bit – některé DSP čipy

Reprezentace hodnot

Uložená čísla mohou reprezentovat různé věci a je na nás, abychom se k nim správně chovali. Zkusme například vzít počítadlo, standardně má 10 kuliček na řádku. Jedna plná řádka je tudíž 10, 2 řádky jsou 20, 3 řádky 30.

My si ale můžeme představit, že jednotlivé řádky jsou „řády“ v naší desítkové soustavě. Jednu kuličku si budeme muset vyndat (či nepoužívat), neboť v desítkové soustavě mohou být pouze hodnoty 0 – 9. Jedna plná řádka pak bude 9, 2 plné řádky 99 a 3 plné řádky 999.

Sčítání

Sčítání na klasickém počítadle funguje tak, že postupně přidáváme požadovaný počet kuliček.

Při sčítání na desítkovém počítadle musíme postupovat po cifrách, čili jednotky sečteme s jednotkami, desítky s desítkami a podobně. Při „přetečení“ (dojdou nám kuličky u jedné cifry) vynulujeme aktuální cifru a zvýšíme cifru vyššího řádu (podobně jako u sčítání pod sebou).

Záporná čísla

Pokud potřebujeme počítat se zápornými čísly (menšími než 0), musíme se domluvit, jak je reprezentovat. Jedna z možností je využít nejvyššího řádu k identifikaci záporného čísla:

$$2 == 0010$$

$$-2 == 1010$$

Nevýhodou je, že máme dvě nuly (0000 a 1111) a počítání nefunguje úplně jednoduše ($0010 + 1010 \Rightarrow 1100$). Proto se spíše používá druhý doplněk, kdy znegujeme (prohodíme 0 za 1 a naopak) hodnotu a přičteme 1:

$$2 == 0010$$

$$-2 == -0010 \rightarrow 1101 + 1 \rightarrow 1110$$

Nyní máme pouze jednu nulu (0000) a aritmetika funguje ($0010 + 1110 \Rightarrow 0000$).

Všimněte si, že použitím nejvyššího řádu jako znaménka omezíme rozsah našeho 4 bitového čísla z 0 - 15 na -8 – 7.

Čísla s plovoucí desetinnou čárkou

(velmi zjednodušené a upravené oproti standardu IEEE)

Pro počítání s opravdu velkými čísly bud' potřebujeme širší registry, nebo můžeme využít méně přesná čísla, která následně vynásobíme určitou hodnotou. Takovým číslům se říká čísla s plovoucí desetinnou čárkou. Matematika s nimi je o něco složitější, ale umožní nám kombinovat různě velká čísla.

Podobně jako u záporných čísel i zde je na nás, jak si číslo rozdělíme. Pokud například máme 8bitové číslo, můžeme si říci, že první 4 číslice využijeme pro „exponent“ a druhé 4 číslice pro vlastní hodnotu, kterou budeme chápat jako číslo s jednou desetinnou čárkou (např. 0010 = 0,2).

0000 0000 → 0,0 * 10⁰ = 0
0000 0010 → 0,2 * 10⁰ = 0,2
0001 0010 → 0,2 * 10¹ = 2
0010 0010 → 0,2 * 10² = 20
0010 0000 → 0,0 * 10² = 0

Oproti obyčejným 8bitovým číslům omezíme rozsah hodnoty z 0–15 (resp. 0,0–1,5), ale můžeme si číslo „posunout“ o 0–15 řádů. Díky tomu můžeme reprezentovat velmi malá i velmi velká čísla, i když jen s omezenou přesností.

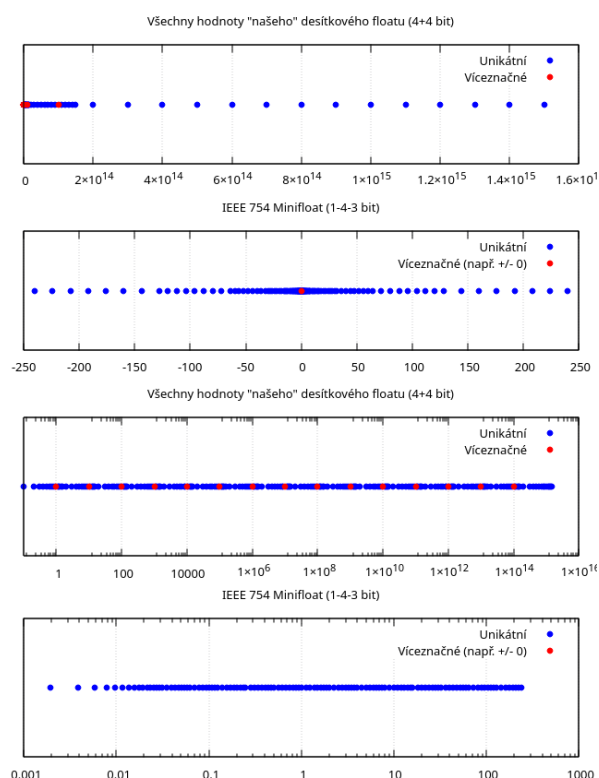
Hodnota tedy není celé číslo, ale číslo s desetinnou čárkou, což nám umožňuje reprezentovat více různých hodnot, nejen násobky deseti.

0,1, 0,2, 0,3, ..., 0,9, 1,0, 2,0, 3,0, ..., 10, 20, 30, ..., 100, 200, 300, ...

Opět bychom mohli využít dvojkový doplněk a uvažovat záporná čísla, případně i záporný exponent pro reprezentaci velmi malých hodnot. Tím bychom mohli vyjádřit čísla například v rozsahu od velmi malých desetinných hodnot až po velká čísla, opět za cenu většího zaokrouhlení.

Ukázky:

0000 0000 → 0,0×10 ⁰ = 0	0001 0000 → 0,0×10 ¹ = 0
0000 0001 → 0,1×10 ⁰ = 0,1	0001 0001 → 0,1×10 ¹ = 1
0000 0010 → 0,2×10 ⁰ = 0,2	0001 0010 → 0,2×10 ¹ = 2
0000 0011 → 0,3×10 ⁰ = 0,3	0001 0011 → 0,3×10 ¹ = 3
0000 0100 → 0,4×10 ⁰ = 0,4	0001 0100 → 0,4×10 ¹ = 4
0000 0101 → 0,5×10 ⁰ = 0,5	0001 0101 → 0,5×10 ¹ = 5
0000 0110 → 0,6×10 ⁰ = 0,6	0001 0110 → 0,6×10 ¹ = 6
0000 0111 → 0,7×10 ⁰ = 0,7	0001 0111 → 0,7×10 ¹ = 7
0000 1000 → 0,8×10 ⁰ = 0,8	0001 1000 → 0,8×10 ¹ = 8
0000 1001 → 0,9×10 ⁰ = 0,9	0001 1001 → 0,9×10 ¹ = 9
0000 1010 → 1,0×10 ⁰ = 1	0001 1010 → 1,0×10 ¹ = 10
0000 1011 → 1,1×10 ⁰ = 1,1	0001 1011 → 1,1×10 ¹ = 11
0000 1100 → 1,2×10 ⁰ = 1,2	0001 1100 → 1,2×10 ¹ = 12
0000 1101 → 1,3×10 ⁰ = 1,3	0001 1101 → 1,3×10 ¹ = 13
0000 1110 → 1,4×10 ⁰ = 1,4	0001 1110 → 1,4×10 ¹ = 14
0000 1111 → 1,5×10 ⁰ = 1,5	0001 1111 → 1,5×10 ¹ = 15



Písmena

Registry mohou obsahovat nejen čísla, ale i písmena. Nejrozšířenějším formátem je kódování ASCII, které se rozšířilo na 8 bitových počítačích. Písmena A-Z jsou definovány jako čísla 65 – 90 bez diakritiky. Celou tabulku si můžeme prohlédnout níže:

0 NUL	NULL character	32 SP	64 @	96 `
1 SOH	Start of Header	33 !	65 A	97 a
2 STX	Start of Text	34 "	66 B	98 b
3 ETX	End of Text	35 #	67 C	99 c
4 EOT	End of Transmission	36 \$	68 D	100 d
5 ENQ	Enquiry	37 %	69 E	101 e
6 ACK	Acknowledge	38 &	70 F	102 f
7 BEL	Bell	39 '	71 G	103 g
8 BS	Backspace	40 (	72 H	104 h
9 HT	Horizontal Tab	41)	73 I	105 i
10 LF	Line feed	42 *	74 J	106 j
11 VT	Vertical Tab	43 +	75 K	107 k
12 FF	Form Feed	44 ,	76 L	108 l
13 CR	Carriage return	45 -	77 M	109 m
14 SO	Shift Out	46 .	78 N	110 n
15 SI	Shift In	47 /	79 O	111 o
16 DLE	Data Link Escape	48 0	80 P	112 p
17 DC1	Device Control (XOn)	49 1	81 Q	113 q
18 DC2	Device Control	50 2	82 R	114 r
19 DC3	Device Control (XOff)	51 3	83 S	115 s
20 DC4	Device Control	52 4	84 T	116 t
21 NAK	Negative Acknowledge	53 5	85 U	117 u
22 SYN	Synchronous Idle	54 6	86 V	118 v
23 ETB	End of Transmission Block	55 7	87 W	119 w
24 CAN	Cancel	56 8	88 X	120 x
25 EM	End of Medium	57 9	89 Y	121 y
26 SUB	Substitute	58 :	90 Z	122 z
27 ESC	Escape	59 ;	91 [	123 {
28 FS	File Separator	60 <	92 \	124
29 GS	Group Separator	61 =	93]	125 }
30 RS	Record Separator	62 >	94 ^	126 ~
31 US	Unit Separator	63 ?	95 _	127 DEL

LF = Line feed = Odřádkování = Enter (někdy nutno LF+CR)

SP = Space = Mezera

Pokud bychom tedy chtěli přechíst hodnotu 4 registrů o kterých víme, že obsahují znaky v ASCII kódu, nahradíme jejich hodnotu odpovídajícím znakem:

01000001, 01001000, 01001111, 01001010, == 65, 72, 79, 74 == AHOJ

Reprezentace stejných hodnot v paměti dle typu

Hex	Binární	Znak (ASCII/CP1250)	Unsigned	Signed	Float (10^x)	Minifloat (1/4/3)	x86 instrukce
0x00	0000 0000	NUL	0	0	$0,0 \times 10^0 = 0$	0	ADD [EAX], AL
0x01	0000 0001	SOH	1	1	$0,1 \times 10^0 = 0,1$	0,001953125	ADD [ECX], AL
0x07	0000 0111	BEL	7	7	$0,7 \times 10^0 = 0,7$	0,013671875	POP ES
0x0A	0000 1010	'\n'	10	10	$1,0 \times 10^0 = 1$	0,03125	OR [EAX], CL
0x20	0010 0000	' '	32	32	$0,0 \times 10^2 = 0$	0,5	AND [EAX], AH
0x30	0011 0000	'0'	48	48	$0,0 \times 10^3 = 0$	1	XOR [EAX], DH
0x41	0100 0001	'A'	65	65	$0,1 \times 10^4 = 1000$	2,25	INC ECX
0x42	0100 0010	'B'	66	66	$0,2 \times 10^4 = 2000$	2,5	INC EDX
0x43	0100 0011	'C'	67	67	$0,3 \times 10^4 = 3000$	2,75	INC EBX
0x61	0110 0001	'a'	97	97	$0,1 \times 10^6 = 100000$	9	(prefix/část)
0x62	0110 0010	'b'	98	98	$0,2 \times 10^6 = 200000$	10	(prefix/část)
0x63	0110 0011	'c'	99	99	$0,3 \times 10^6 = 300000$	11	(prefix/část)
0x78	0111 1000	'x'	120	120	$0,8 \times 10^7$	16	JS rel8
0x08	0000 1000	BS	8	8	$0,8 \times 10^0 = 0,8$	0,015625	OR [EAX], CL
0x9A	1001 1010	'š' (CP1250)	154	-102	$1,0 \times 10^9$	-0,625	CALL ptr16:16/32
0xE1	1110 0001	'á' (CP1250)	225	-31	$0,1 \times 10^{14} = 10^{13}$	-2,25	LOOPZ rel8
0xEB	1110 1011	ě (CP1250)	235	-21	$1,1 \times 10^{14}$	-6,5	JMP rel8
0x7F	0111 1111	DEL	127	127	$1,5 \times 10^7$	240	JG rel8
0xFF	1111 1111	ÿ (CP1250)	255	-1	$1,5 \times 10^{15}$	NaN	(skupinová)

Pracovní list

Převod z desítkové do dvojkové

a) 3 = _____

b) 5 = _____

c) 6 = _____

d) 9 = _____

e) 10 = _____

f) 12 = _____

g) 14 = _____

h) 15 = _____

Převod z dvojkové do desítkové

a) 10 = _____

b) 11 = _____

c) 101 = _____

d) 111 = _____

e) 1000 = _____

f) 1001 = _____

g) 1010 = _____

h) 1111 = _____

Rozklad čísla (pomocí hodnot 1, 2, 4, 8, 16)

Příklad:

$$1011 = 8 + 0 + 2 + 1 = 11$$

a) 1101 = _____ + _____ + _____ + _____ = _____

b) 1010 = _____ + _____ + _____ + _____ = _____

c) 10011 = _____ + _____ + _____ + _____ + _____ = _____

d) 1110 = _____ + _____ + _____ + _____ = _____

e) 10101 = _____ + _____ + _____ + _____ + _____ = _____

f) 11000 = _____ + _____ + _____ + _____ + _____ = _____

g) 11111 = _____ + _____ + _____ + _____ + _____ = _____

h) 10010 = _____ + _____ + _____ + _____ + _____ = _____

Sčítání v dvojkové soustavě

a)

0001

+ 0001

c)

0111

+ 0001

e)

0110

+ 0010

g)

1110

+ 0010

b)

0010

+ 0011

d)

0101

+ 0001

f)

1001

+ 0001

h)

1011

+ 0101

Ascii kód

65 -> ____ 66 -> ____ 67 -> ____ 68 -> ____ 69 -> ____ 70 -> ____

Výsledek: _____

80 -> ____ 79 -> ____ 67 -> ____ 73 -> ____ 84 -> ____ 64 -> ____ 67 -> ____

Výsledek: _____

Binární obrázky

1;8									
1;8									
3;12									
7;14									
15;15									
9;9									
1;8									
3;12									

7;14									
8;1									
10;5									
8;1									
10;5									
9;9									
8;1									
7;14									

0;0									
6;6									
9;9									
9;9									
4;2									
2;4									
1;8									
0;0									

1;4									
0;8									
1;4									
0;8									
0;8									
3;14									
1;12									
1;12									

